Anatomy of Lisp

John Allen

Preface

Chapter 1 Symbolic Expressions

1.1 Introduction
1.2 Symbolic Expressions: Abstract Data Structures
1.3 Trees: Representations of Symbolic Expressions
1.4 Primitive Functions
1.5 Predicates and Conditional Expressions
1.6 Sequences: Abstract Data Structures
1.7 Lists: Representations of Sequences
1.8 A Respite
1.9 Becoming an Expert

Chapter 2 Applications of Lisp

2.1 Introduction
2.2 Examples of Lisp Applications
2.3 Differentiation
2.4 Tree Searching
2.5 Data Bases
2.6 Algebra of Polynomials
2.7 Evaluation of Polynomials
2.8 The Great Progenitors
2.9 Another Respite
2.10 Proving Properties of Programs

Chapter 3 Evaluation of Lisp Expressions

3.1 Introduction
3.2 S-expr Translation of Lisp Expressions
3.3 Symbol Tables
3.4 λ-notation

3.5 Mechanization of Evaluation
3.6 Examples of *eval*
3.7 Variables
3.8 Environments and Bindings
3.9 *label*
3.10 Functional Arguments and Functional Values
3.11 Binding Strategies and Implementations
3.12 Special Forms and Macros
3.13 Review and Reflection

Chapter 4 Imperative Constructs in Lisp

4.1 Introduction
4.2 The *prog*-feature
4.3 Alternatives to *prog*
4.4 Extensions to *eval*
4.5 Non-recursive Control Structures
4.6 *eval* with Explicit Access
4.7 *eval* with Explicit Control
4.8 An Evaluator for *prog*
4.9 Alternatives to *eval*
4.10 Function Definitions
4.11 Rapprochement: In Retrospect
4.12 Lisp Machines

Chapter 5 The Static Structure of Lisp

5.1 Introduction
5.2 Representation of S-expressions
5.3 Representation of Lisp Primitives
5.4 AMBIT/G
5.5 A Few Programming Techniques
5.6 Symbol Tables and Property-lists
5.7 Property-list Functions
5.8 An *eval* for Property-lists

5.9 Representation of Property-lists
5.10 A Picture of the Atom *NIL*
5.11 Input/Output: *read* and *print*
5.12 Table Searching: Hashing
5.13 A First Look at *cons*
5.14 Storage Management: Garbage Collection
5.15 A Simple Lisp Garbage Collector
5.16 A Review of the Structure of the Lisp Machine
5.17 Implementations of Binding
5.18 Stack Implementation of a Lisp Subset: Deep Bound
5.19 Stack Implementation of a Lisp Subset: Shallow Bound
5.20 Strategies for Full Lisp Implementation
5.21 Epilogue

Chapter 6 The Dynamic Structure of Lisp

6.1 Introduction
6.2 Primitives of Lisp
6.3 **SM**: A Simple Machine
6.4 Implementation of the Primitives
6.5 Assemblers
6.6 Compilers for Subsets of Lisp
6.7 Compilation of Conditional Expressions
6.8 One-pass Assemblers and Fixups
6.9 A Compiler for Simple *eval*: The Value Stack
6.10 A Compiler for Simple *eval*
6.11 Efficient Compilation
6.12 Efficiency: Primitive Operations
6.13 Efficiency: Calling Sequences
6.14 Efficiency: Predicates
6.15 A Compiler for *progs*
6.16 Further Optimizations
6.17 Functional Arguments
6.18 Macros and Special Forms

6.19 Compilation and Variables
6.20 Compiling and Interpreting
6.21 Interactive Programming
6.22 Lisp Editors
6.23 Debugging in Lisp

Chapter 7 Storage Structures and Efficiency

7.1 Introduction
7.2 Vectors and Arrays
7.3 Strings and Linear Lisp
7.4 A Compacting Collector for Lisp
7.5 Bit-tables
7.6 Representations of Complex Data Structures
7.7 *rplaca* and *rplacd*
7.8 Applications of *rplaca* and *rplacd*
7.9 Numbers
7.10 Stacks and Threading
7.11 A Non-recursive *read*
7.12 More Applications of Threading
7.13 Storage Management and Lisp
7.14 Hash Techniques

Chapter 8 Implications of Lisp

Chapter 9 Projects

9.1 Extensions to *eval*
9.2 Pretty-printing
9.3 Syntax-directed Processes
9.4 Syntax-directed I/O

Bibliography

Index